

Федеральное государственное автономное образовательное учреждение
высшего образования

Национальный исследовательский университет ИТМО

Факультет программной инженерии и компьютерной техники

Направление подготовки 09.03.04 Программная инженерия

Дисциплина «Распределённые системы хранения данных»

Отчёт

по лабораторной работе №2

Вариант: 70219

Студент:

Чусовлянов Максим Сергеевич

Садовой Григорий Владимирович

Группа Р3307

Преподаватель:

Максимов Андрей Николаевич

г. Санкт-Петербург, 2026 г.

Оглавление

Задание	3
Этап 1. Резервное копирование	5
Настройка SSH-ключа	5
Скрипт резервного копирования	5
Пробный запуск	6
Расписание стоп	6
Расчёт объёма резервных копий за 30 дней	7
Этап 2. Потеря основного узла	9
Описание сценария	9
Скрипт восстановления	9
Выполнение	10
Проверка данных	10
Этап 3. Повреждение файлов БД	11
Описание сценария	11
Симуляция сбоя	11
Скрипт восстановления	11
Выполнение	12
Проверка данных	12
Этап 4. Логическое повреждение данных (WAL PITR)	13
Настройка архивирования WAL	13
Вставка строк и симуляция ошибки	13
Восстановление через PITR	14
Ход восстановления (из лога)	15
Проверка результата	15
Исходный код	17
Вывод	18

Задание

Цель работы - настроить процедуру периодического резервного копирования базы данных, сконфигурированной в ходе выполнения лабораторной работы №2, а также разработать и отладить сценарии восстановления в случае сбоев.

Узел из предыдущей лабораторной работы используется в качестве основного. Новый узел используется в качестве резервного. Учётные данные для подключения к новому узлу выдаёт преподаватель. В сценариях восстановления необходимо использовать копию данных, полученную на первом этапе данной лабораторной работы.

Требования к отчёту

Отчет должен быть самостоятельным документом (без ссылок на внешние ресурсы), содержать всю последовательность команд и исходный код скриптов по каждому пункту задания. Для демонстрации результатов приводить команду вместе с выводом (самой наглядной частью вывода, при необходимости).

Этап 1. Резервное копирование

Настроить резервное копирование с основного узла на резервный следующим образом: периодические холодные полные копии. Полная копия (rsync) по расписанию (cron) раз в сутки. СУБД на время копирования должна останавливаться. На резервном узле хранить 14 копий, после успешного создания пятнадцатой копии, самую старую автоматически уничтожать.

Подсчитать, каков будет объем резервных копий спустя месяц работы системы, исходя из следующих условий: средний объем новых данных в БД за сутки: 650МБ; средний объем измененных данных за сутки: 50МБ. Проанализировать результаты.

Этап 2. Потеря основного узла

Этот сценарий подразумевает полную недоступность основного узла. Необходимо восстановить работу СУБД на РЕЗЕРВНОМ узле, продемонстрировать успешный запуск СУБД и доступность данных.

Этап 3. Повреждение файлов БД

Этот сценарий подразумевает потерю данных (например, в результате сбоя диска или файловой системы) при сохранении доступности основного узла. Необходимо выполнить полное восстановление данных из резервной копии и перезапустить СУБД на ОСНОВНОМ узле.

Ход работы: симулировать сбой — удалить с диска директорию конфигурационных файлов СУБД со всем содержимым. Проверить работу СУБД, доступность данных, перезапустить СУБД, проанализировать результаты. Выполнить восстановление данных из резервной копии, учитывая следующее условие: исходное расположение дополнительных табличных пространств недоступно — разместить в другой директории и скорректировать конфигурацию. Запустить СУБД, проверить работу и доступность данных, проанализировать результаты.

Этап 4. Логическое повреждение данных

Этот сценарий подразумевает частичную потерю данных (в результате нежелательной или ошибочной операции) при сохранении доступности основного узла. Необходимо выполнить восстановление данных на ОСНОВНОМ узле следующим способом: восстановление с использованием архивных WAL файлов (СУБД должна работать в режиме архивирования WAL, потребуется задать параметры восстановления).

Ход работы: в каждую таблицу базы добавить 2–3 новые строки, зафиксировать результат. Зафиксировать время и симулировать ошибку — удалить любые две таблицы (DROP TABLE). Продемонстрировать результат. Выполнить восстановление данных указанным способом. Продемонстрировать и проанализировать результат.

Этап 1. Резервное копирование

Настройка SSH-ключа

Для прямого rsync из pg116 в pg117 без ввода пароля генерируем ключ на основном узле и добавляем его в authorized_keys на резервном:

```
# на pg116
ssh-keygen -t ed25519 -N "" -f ~/.ssh/id_ed25519
# Публичный ключ:
# ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIN6w50jswEpiZBkR45HaT/
# 8Qehf/joL/TRhx5JDFS64X postgres1@pg116.cs.ifmo.ru

# на pg117
echo "ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIN6w50jswEpiZBkR45HaT/8Qeh\
f/joL/TRhx5JDFS64X postgres1@pg116.cs.ifmo.ru" >> ~/.ssh/authorized_keys
chmod 600 ~/.ssh/authorized_keys
```

Проверка прямого соединения:

```
postgres1@pg116$ ssh postgres4@pg117 hostname
pg117.cs.ifmo.ru
```

Скрипт резервного копирования

```
#!/bin/sh
set -e

PGDATA="$HOME/aoj42"
REMOTE="postgres4@pg117"
STAMP="$(date +%Y-%m-%d-%H-%M-%S)"
LOGFILE="$HOME/backup.log"

log() { echo "$(date '+%Y-%m-%d %H:%M:%S') $*" | tee -a "$LOGFILE"; }

log "=== начало бэкапа: $STAMP ==="

REMOTE_HOME="$(ssh "$REMOTE" 'echo $HOME')"
DEST="$REMOTE_HOME/backups/backup-$STAMP"
log "назначение: $DEST"

log "останавливаем postgres"
pg_ctl -D "$PGDATA" stop -m fast

ssh "$REMOTE" "mkdir -p '$DEST/pgdata' '$DEST/ts_ydp10' '$DEST/ts_zcc31'"

log "rsync pgdata"
rsync -az --delete "$PGDATA/" "$REMOTE:$DEST/pgdata/"

log "rsync ts_ydp10"
rsync -az --delete "$HOME/ydp10/" "$REMOTE:$DEST/ts_ydp10/"
```

```
log "rsync ts_zcc31"
rsync -az --delete "$HOME/zcc31/" "$REMOTE:$DEST/ts_zcc31/"

log "запускаем postgres"
pg_ctl -D "$PGDATA" -l "$PGDATA/server.log" start

# оставляем 14 копий, удаляем лишние
ssh "$REMOTE" 'cd $HOME/backups && ls -dt backup-* 2>/dev/null \
| tail -n +15 | xargs -r rm -rf'

log "=== готово: $DEST ==="
```

Скрипт загружается в ~/scripts/backup.sh на pg116, права chmod +x.

Пробный запуск

```
postgres1@pg116$ ~/scripts/backup.sh
2026-05-19 08:24:39 === начало бэкапа: 2026-05-19-08-24-39 ===
2026-05-19 08:24:39 назначение: /var/db/postgres4/backups/
backup-2026-05-19-08-24-39
2026-05-19 08:24:39 останавливаем postgres
ожидание завершения работы сервера..... готово
сервер остановлен
2026-05-19 08:24:40 rsync pgdata
2026-05-19 08:24:40 rsync ts_ydp10
2026-05-19 08:24:41 rsync ts_zcc31
2026-05-19 08:24:41 запускаем postgres
ожидание запуска сервера..... готово
сервер запущен
2026-05-19 08:24:41 === готово: /var/db/postgres4/backups/
backup-2026-05-19-08-24-39 ===
```

Проверка содержимого бэкапа на pg117:

```
postgres4@pg117$ ls ~/backups/backup-2026-05-19-08-24-39/
pgdata  ts_ydp10  ts_zcc31

postgres4@pg117$ du -sh ~/backups/backup-2026-05-19-08-24-39/*
13M  pgdata
7.5K ts_ydp10
7.5K ts_zcc31
```

Расписание cron

```
# на pg116
crontab -e
# добавить строку:
0 3 * * * /var/db/postgres1/scripts/backup.sh >> /var/db/postgres1/backup.log
2>&1

postgres1@pg116$ crontab -l
0 3 * * * /var/db/postgres1/scripts/backup.sh >> /var/db/postgres1/backup.log
2>&1
```

Резервное копирование запускается ежедневно в 03:00.

Расчёт объёма резервных копий за 30 дней

Исходные данные:

- Начальный размер БД: $S_0 = 30$ МБ (pgdata + оба табличных пространства)
- Прирост новых данных: $\Delta_{\text{new}} = 650$ МБ/сут
- Прирост мёртвых кортежей (autovacuum=off): $\Delta_{\text{dead}} = 50$ МБ/сут
- Суммарный прирост: $\delta = 700$ МБ/сут
- Политика хранения: 14 копий

Размер копии в день d :

$$S(d) = S_0 + d \times \delta$$

Политика ротации. После создания 15-й копии самая старая удаляется. На 30-й день хранятся копии дней 17–30 (14 штук).

Суммарный объём:

$$V = \sum_{d=17}^{30} S(d) = 14 \cdot S_0 + \delta \times \sum_{d=17}^{30} d = 14 \times 30 \text{ МБ} + 700 \times 329 \text{ МБ}$$

$$V \approx 420 \text{ МБ} + 230300 \text{ МБ} \approx 225 \text{ ГБ}$$

День	Размер копии	Накопленный итог
17	11 930 МБ	11 930 МБ
18	12 630 МБ	24 560 МБ
19	13 330 МБ	37 890 МБ
20	14 030 МБ	51 920 МБ
21	14 730 МБ	66 650 МБ
22	15 430 МБ	82 080 МБ
23	16 130 МБ	98 210 МБ
24	16 830 МБ	115 040 МБ
25	17 530 МБ	132 570 МБ
26	18 230 МБ	150 800 МБ
27	18 930 МБ	169 730 МБ
28	19 630 МБ	189 360 МБ
29	20 330 МБ	209 690 МБ
30	21 030 МБ	230 720 МБ

Анализ. Холодный rsync без сжатия и дедупликации — наиболее затратный по дисковому пространству метод: каждая копия содержит полный образ БД, включая мёртвые кортежи (autovacuum отключён). При 225 ГБ суммарного объёма на 30-й день потребуется канал пропускной способностью не менее $\frac{S(d)}{T_{\text{окно}}}$; при 4-часовом ночном

окне это около 4 ГБ/ч. Преимущество — простота и надёжность восстановления: достаточно скопировать файлы обратно.

Этап 2. Потеря основного узла

Описание сценария

Основной узел pg116 недоступен. Задача — поднять СУБД на резервном узле pg117 из последнего rsync-бэкапа.

Скрипт восстановления

```
#!/bin/sh
set -e

PGDATA="$HOME/aoj42"
PGPORT=9748
PGUSER=postgres0
TS_YDP10="$HOME/ts_ydp10"
TS_ZCC31="$HOME/ts_zcc31"

LATEST="$(ls -dt "$HOME/backups"/backup-* | head -1)"
echo "бэкап: $LATEST"

pg_ctl -D "$PGDATA" stop -m fast 2>/dev/null || true

rm -rf "$PGDATA" "$TS_YDP10" "$TS_ZCC31"
mkdir -p "$PGDATA" "$TS_YDP10" "$TS_ZCC31"
chmod 700 "$PGDATA"

rsync -a "$LATEST/pgdata/" "$PGDATA/"
rsync -a "$LATEST/ts_ydp10/" "$TS_YDP10/"
rsync -a "$LATEST/ts_zcc31/" "$TS_ZCC31/"

# правим симлинки табличных пространств
for link in "$PGDATA/pg_tblspc/"*; do
    [ -L "$link" ] || continue
    case "$(readlink "$link")" in
        */ydp10) ln -sf "$TS_YDP10" "$link" ;;
        */zcc31) ln -sf "$TS_ZCC31" "$link" ;;
    esac
done

ls -la "$PGDATA/pg_tblspc/"

# убираем путь к сокету pg116 – на pg117 он другой
grep -v unix_socket_directories "$PGDATA/postgresql.auto.conf" \
    > "$PGDATA/postgresql.auto.conf.tmp" 2>/dev/null || true
mv "$PGDATA/postgresql.auto.conf.tmp" "$PGDATA/postgresql.auto.conf"

cat >> "$PGDATA/postgresql.conf" <<EOF
listen_addresses = 'localhost'
port = $PGPORT
```

```
shared_buffers = '128MB'
autovacuum = off
unix_socket_directories = '$HOME/run'
EOF
```

```
mkdir -p "$HOME/run"
pg_ctl -D "$PGDATA" -l "$PGDATA/server.log" start
```

Выполнение

```
postgres4@pg117$ ~/restore.sh
бэкап: /var/db/postgres4/backups/backup-2026-05-19-08-28-58
total 2
lrwx----- 16385 -> /var/db/postgres1/ypd10
lrwx----- 16386 -> /var/db/postgres1/zcc31
симвлики после:
lrwxr-xr-x 16385 -> /var/db/postgres4/ts_ypd10
lrwxr-xr-x 16386 -> /var/db/postgres4/ts_zcc31
ожидание запуска сервера.... готово
сервер запущен
```

Проверка данных

```
postgres4@pg117$ psql -h ~/run -p 9748 -U postgres0 -l
```

Имя	Владелец
bestbluemath	postgres0
postgres	postgres0
template0	postgres0
template1	postgres0

```
postgres4@pg117$ psql -h ~/run -p 9748 -U postgres0 -d bestbluemath \
```

```
-c "SELECT 'data_default' AS tbl, count(*) FROM data_default
      UNION ALL SELECT 'data_ypd10', count(*) FROM data_ypd10
      UNION ALL SELECT 'data_zcc31', count(*) FROM data_zcc31"
```

tbl	count
data_default	200
data_ypd10	200
data_zcc31	200

Анализ. Холодная копия rsync при условии корректной остановки СУБД перед бэкапом содержит согласованный образ данных. Ключевой шаг при переносе на новый хост — исправление симлинков pg_tblspc/<OID> и корректировка unix_socket_directories. Время восстановления зависит только от скорости rsync и размера данных.

Этап 3. Повреждение файлов БД

Описание сценария

Основной узел pg116 доступен, но конфигурационные файлы СУБД утрачены. Исходное расположение дополнительных табличных пространств также недоступно — их нужно перенести в новую директорию.

Симуляция сбоя

```
# Останавливаем сервер
pg_ctl -D ~/aoj42 stop -m fast
# Удаляем конфигурационные файлы
rm -f ~/aoj42/postgresql.conf ~/aoj42/postgresql.auto.conf ~/aoj42/pg_hba.conf
# Пытаемся запустить — ожидаем ошибку
pg_ctl -D ~/aoj42 start
postgres1@pg116$ pg_ctl -D ~/aoj42 start
ожидание запуска сервера.... прекращение ожидания
pg_ctl: не удалось запустить сервер
postgres: не удалось открыть файл конфигурации
"/var/db/postgres1/aoj42/postgresql.conf": No such file or directory
```

Скрипт восстановления

```
#!/bin/sh
set -e

PGDATA="$HOME/aoj42"
REMOTE="postgres4@pg117"
LATEST="$(ssh "$REMOTE" 'ls -dt $HOME/backups/backup-* | head -1')"
```

echo "восстанавливаем из \$LATEST"

```
rsync -a "$REMOTE:$LATEST/pgdata/postgresql.conf" "$PGDATA/"
rsync -a "$REMOTE:$LATEST/pgdata/postgresql.auto.conf" "$PGDATA/"
rsync -a "$REMOTE:$LATEST/pgdata/pg_hba.conf" "$PGDATA/"
```

табличные пространства переносим в новое место

```
mkdir -p "$HOME/ts_restore/ydp10" "$HOME/ts_restore/zcc31"
rsync -a "$REMOTE:$LATEST/ts_ydp10/" "$HOME/ts_restore/ydp10/"
rsync -a "$REMOTE:$LATEST/ts_zcc31/" "$HOME/ts_restore/zcc31/"
```

echo "символические ссылки до:"

```
ls -la "$PGDATA/pg_tblspc/"
```

```
for link in "$PGDATA/pg_tblspc/"*; do
[ -L "$link" ] || continue
case "$(readlink "$link")" in
*/ydp10) ln -sfn "$HOME/ts_restore/ydp10" "$link" ;;
*/zcc31) ln -sfn "$HOME/ts_restore/zcc31" "$link" ;;
esac
```

done

```
echo "симвлики после:"
```

```
ls -la "$PGDATA/pg_tblspc/"
```

```
pg_ctl -D "$PGDATA" -l "$PGDATA/server.log" start
```

Выполнение

```
postgres1@pg116$ ~/scripts/02-restore.sh
```

```
восстанавливаем из /var/db/postgres4/backups/backup-2026-05-19-08-28-58
```

```
симвлики до:
```

```
lrwxr-xr-x 16385 -> /var/db/postgres1/ypd10
```

```
lrwxr-xr-x 16386 -> /var/db/postgres1/zcc31
```

```
симвлики после:
```

```
lrwxr-xr-x 16385 -> /var/db/postgres1/ts_restore/ypd10
```

```
lrwxr-xr-x 16386 -> /var/db/postgres1/ts_restore/zcc31
```

```
ожидание запуска сервера.... готово
```

```
сервер запущен
```

Проверка данных

```
postgres1@pg116$ psql -h ~/run -p 9748 -U postgres0 -d bestbluemath \
```

```
-c "SELECT 'data_default' AS tbl, count(*) FROM data_default
      UNION ALL SELECT 'data_ypd10', count(*) FROM data_ypd10
      UNION ALL SELECT 'data_zcc31', count(*) FROM data_zcc31"
```

tbl	count
data_default	200
data_ypd10	200
data_zcc31	200

Анализ. Утрата конфигурационных файлов не затрагивает файлы данных (base, global, pg_tblspc), поэтому достаточно восстановить только три файла конфигурации. При переносе табличных пространств необходимо скорректировать символы pg_tblspc/<OID> — именно через них PostgreSQL определяет физическое расположение таблиц. Перезапись данных из бэкапа не требуется.

Этап 4. Логическое повреждение данных (WAL PITR)

Настройка архивирования WAL

Перед любыми изменениями данных включаем архивирование WAL и делаем базовый снимок:

```
ALTER SYSTEM SET wal_level = 'replica';
ALTER SYSTEM SET archive_mode = 'on';
ALTER SYSTEM SET archive_command =
    'test ! -f /var/db/postgres1/wal_archive/%f
    && cp %p /var/db/postgres1/wal_archive/%f';
mkdir -p ~/wal_archive ~/base_backup
pg_ctl -D ~/aoj42 restart

# Разрешить репликационное подключение для pg_basebackup
echo "local replication postgres0 trust" >> ~/aoj42/pg_hba.conf
pg_ctl -D ~/aoj42 reload
```

Базовая резервная копия

```
pg_basebackup -D ~/base_backup -h ~/run -p 9748 \
    -U postgres0 -Ft -Xs -z -P
postgres1@pg116$ ls -lh ~/base_backup/
-rw----- 16385.tar.gz      2.2K  (табличное пространство ts_ydp10)
-rw----- 16386.tar.gz      2.2K  (табличное пространство ts_zcc31)
-rw----- backup_manifest  182K
-rw----- base.tar.gz      4.0M  (основной кластер)
-rw----- pg_wal.tar.gz    17K
```

Проверка параметров:

```
postgres1@pg116$ psql -h ~/run -p 9748 -U postgres0 -d postgres \
    -c "SHOW wal_level; SHOW archive_mode; SHOW archive_command"
wal_level | archive_mode | archive_command
-----+-----+-----
replica  | on           | test ! -f /var/db/postgres1/wal_archive/%f
          |              | && cp %p /var/db/postgres1/wal_archive/%f
```

Вставка строк и симуляция ошибки

```
-- Вставляем тестовые строки в каждую таблицу
INSERT INTO data_ydp10 (payload) VALUES ('pitr_row_1'), ('pitr_row_2'),
('pitr_row_3');
INSERT INTO data_zcc31 (payload) VALUES ('pitr_row_1'), ('pitr_row_2'),
('pitr_row_3');
INSERT INTO data_default(payload) VALUES ('pitr_row_1'), ('pitr_row_2'),
('pitr_row_3');

-- Фиксируем время (цель восстановления — ДО удаления таблиц)
SELECT now() AS recovery_target_time;
```

```

        recovery_target_time
-----
2026-05-19 08:26:12.280237+03
-- Переключаем WAL-сегмент (гарантируем архивацию всех изменений)
SELECT pg_switch_wal();
-- Симулируем ошибку – удаляем две таблицы
DROP TABLE data_ydp10;
DROP TABLE data_zcc31;
postgres1@pg116$ psql -h ~/run -p 9748 -U postgres0 -d bestbluemath -c "\dt"
      Имя          |  Тип   | Владелец
-----+-----+-----
data_default | таблица | postgres0
(1 строка)

```

Восстановление через PITR

```

pg_ctl -D ~/aoj42 stop -m fast

# Очищаем PGDATA
rm -rf ~/aoj42
mkdir ~/aoj42
chmod 700 ~/aoj42

# Распаковываем базовую копию
tar -xzf ~/base_backup/base.tar.gz -C ~/aoj42/

# Восстанавливаем табличные пространства
rm -rf ~/ydp10 ~/zcc31
mkdir -p ~/ydp10 ~/zcc31
tar -xzf ~/base_backup/16385.tar.gz -C ~/ydp10/
tar -xzf ~/base_backup/16386.tar.gz -C ~/zcc31/

# Воссоздаём симлинки pg_tblspc (удалены при rm -rf aoj42)
ln -sf ~/ydp10 ~/aoj42/pg_tblspc/16385
ln -sf ~/zcc31 ~/aoj42/pg_tblspc/16386

# Добавляем параметры восстановления в postgresql.conf
cat >> ~/aoj42/postgresql.conf <<'EOF'
restore_command = 'cp /var/db/postgres1/wal_archive/%f %p'
recovery_target_time = '2026-05-19 08:26:12+03'
recovery_target_action = 'promote'
EOF

# Сигнальный файл – запустить в режиме восстановления
touch ~/aoj42/recovery.signal

pg_ctl -D ~/aoj42 -l ~/aoj42/server.log start

```

Ход восстановления (из лога)

```
LOG: starting point-in-time recovery to 2026-05-19 08:26:12+03
LOG: starting backup recovery with redo LSN 0/2000028
LOG: restored log file "000000010000000000000002" from archive
LOG: redo starts at 0/2000028
LOG: restored log file "000000010000000000000003" from archive
LOG: restored log file "000000010000000000000004" from archive
LOG: recovery stopping before commit of transaction 749,
      time 2026-05-19 08:26:12.203698+03
LOG: redo done at 0/3001B40
LOG: selected new timeline ID: 2
LOG: archive recovery complete
LOG: database system is ready to accept connections
```

Проверка результата

```
postgres1@pg116$ psql -h ~/run -p 9748 -U postgres0 -d postgres \
-c "SELECT pg_is_in_recovery()"
pg_is_in_recovery
```

```
-----
f
```

```
postgres1@pg116$ psql -h ~/run -p 9748 -U postgres0 -d bestbluemath \
-c "\dt" \
-c "SELECT 'data_default' AS tbl, count(*) FROM data_default
      UNION ALL SELECT 'data_ydp10', count(*) FROM data_ydp10
      UNION ALL SELECT 'data_zcc31', count(*) FROM data_zcc31"
```

Список отношений

public	data_default	таблица	postgres0
public	data_ydp10	таблица	postgres0
public	data_zcc31	таблица	postgres0

tbl	count
data_default	200
data_ydp10	200
data_zcc31	200

Анализ. Цель восстановления — 2026-05-19 08:26:12+03 — была установлена в момент между вставкой строк и удалением таблиц. PostgreSQL остановил воспроизведение WAL **перед** транзакцией 749 (время коммита 08:26:12.203), так как тот оказался позже целевой метки. В результате:

- таблицы data_ydp10 и data_zcc31 восстановлены (DROP не применился);
- вставленные строки не вошли в результат (их коммит тоже позже цели);
- pg_is_in_recovery() = f подтверждает, что кластер переведён в нормальный режим (recovery_target_action = promote).

WAL PITR — единственный из рассмотренных методов, позволяющий восстановить состояние на произвольный момент времени без остановки основного сервера в момент сбоя.

Исходный код

Скрипты лабораторной работы:

- `scripts/task1/backup.sh` — холодный `rsync`-бэкап с ротацией 14 копий
- `scripts/task2/restore-on-backup.sh` — восстановление СУБД на резервном узле
- `scripts/task3/01-simulate.sh` — симуляция потери конфигурационных файлов
- `scripts/task3/02-restore.sh` — восстановление конфигурации и переезд TS
- `scripts/task4/01-config.sh` — включение WAL archiving и `pg_basebackup`
- `scripts/task4/02-insert.sh` — вставка строк перед PITR
- `scripts/task4/03-error.sh` — фиксация WAL и DROP TABLE
- `scripts/task4/04-recovery.sh` — PITR-восстановление
- `scripts/task4/05-end-recovery.sh` — завершение recovery, очистка параметров

Вывод

В ходе лабораторной работы была настроена и проверена полная цепочка резервного копирования и восстановления PostgreSQL 16.4:

- **Этап 1:** холодный rsync-бэкап по cron обеспечивает ежедневную полную копию кластера на резервном узле. Расчёт показал, что за 30 дней при хранении 14 копий потребуется 225 ГБ при темпе роста 700 МБ/сут.
- **Этап 2:** при полной недоступности основного узла СУБД была успешно поднята на pg117 из rsync-бэкапа. Ключевые операции — исправление симлинков `pg_tblspc` и перезапись `unix_socket_directories`.
- **Этап 3:** после симулированной потери конфигурационных файлов СУБД не запустилась. Восстановление потребовало только трёх конфигурационных файлов из бэкапа и переноса табличных пространств в новую директорию без копирования данных.
- **Этап 4:** PITR через архивные WAL-файлы позволил восстановить состояние БД на точную временную метку, отменив DROP TABLE без каких-либо потерь в остальных данных.